TASK 1: Understand the Problem and Identify the Objects

Name: K.V.N.D.P.SAI RUPESH | Register Number: 1972260102 | Java Fundamentals and OOP - Assignment 1

Inputs and outputs

- Inputs: vehicle type (two-wheeler, car, EV), registration number, entry time, exit time, whether the vehicle holds a valid monthly pass, and charging units for EVs.

- Outputs: a parking ticket with a unique number and a slot, a rejection when the facility is full or the input is invalid, the parking fee on exit, and the day's total collection.

Candidate classes and why each is needed

Class	Why it is needed	Responsibility
Vehicle (TwoWheeler, Car, ElectricCar)	Each vehicle has a registration number and a rate that depends on its type	Knows its registration number and rate; calculates its own fee
ParkingTicket	The entry record must be kept and linked to one slot and one vehicle	Holds the unique ticket number, vehicle, floor, slot and entry time
ParkingFacility	Floors and slots belong together and need one owner	Allocates a free slot, releases a slot, tracks active tickets
Pass (MonthlyPass, DayPass)	Free parking rules are different from fee rules	Says whether a stay is covered by a pass
Chargeable (interface)	Only EVs pay for charging, so it is a separate capability	Gives the charging fee

Verbs become methods

- enter (a vehicle arrives) becomes allocate() and creates a ticket.

- exit (a vehicle leaves) becomes release() plus calculateFee().

- The fee rule becomes calculateFee(minutes), and the pass rule becomes covers(entry, exit).

Business rules every valid operation must satisfy

- A slot holds at most one vehicle.

- A registration number can be active only once at a time.

- A ticket number is never reused (a static counter gives each new ticket the next number).

- Exit time cannot be earlier than entry time.

- A slot can be released only once; a wrong or empty position returns false.

- A valid pass holder pays zero parking fee, but charging is still added.

Assumptions

- Times are minutes after midnight, and a stay may cross midnight.

- Every started hour is charged in full.

- The facility has fixed slots and no reservations.

- Only the three vehicle types in the scenario exist (the e-bike and van come in Task 12).

Alternative concept: parallel arrays instead of objects

Instead of objects, each vehicle could be stored as one index across three arrays. The same index describes one vehicle.

```
String[] regNo = new String[100];
String[] type = new String[100];
int[] entryTime = new int[100];
int count = 0;
```

Finding a vehicle by registration number needs a loop over regNo, so the search takes O(n) time. Adding a new attribute such as pass status means a new array and a change to every place that adds or removes a vehicle, and a missed change puts the arrays out of step. Space per vehicle is a few references and values with no per-object header, so the arrays are slightly smaller. The object design costs a little more memory per vehicle, but keeps each vehicle's data together and makes adding an attribute a one-line change in one class.